# Project 2: Architecture Simulation

ECE/CSC 406/506, Spring 2026

The **Structural Simulation Toolkit (SST)** is a parallel simulation framework used for modeling and analyzing high-performance computing systems. It allows detailed simulation of components such as processors, memory hierarchies, interconnects, and coherence protocols.

SST is organized into two main parts:

- **SST Core**: Provides the simulation engine, event handling, and infrastructure.
- **SST Elements**: A collection of component models (e.g., CPUs, caches, memory controllers) built on top of the core. The memory hierarchy and coherence protocols used in this assignment are part of SST Elements.

In this assignment, you will work with the **memHierarchy** component, specifically the L1 cache coherence manager implementing the MESI protocol.

Some useful references to learn more about SST:

1. http://sst-simulator.org/sst-docs/docs/elements/memHierarchy/intro
2. http://sst-simulator.org/sst-docs/docs/elements/memHierarchy/cache
3. http://sst-simulator.org/sst-docs/docs/elements/miranda/intro
4. https://people.cs.pitt.edu/~childers/CS2410/sst.pdf

The assignment will be submitted in two separate stages.

1. 406 and 506: Collect memory statistics for multiprocessor address traces
   **Due: April 3**

2. 506 only: Implement and demonstrate a coherence optimization
   **Due: April 17**

## Part 1: Memory Statistics

In this assignment, you will add instrumentation to the L1 MESI coherence manager such that a set of counters defined in `Counters.h` are incremented whenever the corresponding MESI coherence events occur.

### *Objectives:*

- Read and understand how coherence state transitions are handled in `MESI_L1` (L1 cache-side protocol logic) for both MSI and MESI protocols.

- Identify the exact points where requests are sent and where protocol state transitions occur.
- Update `MESI_L1` to increment the appropriate counters without modifying the functional behavior of the coherence protocol. Use cachename_ variable for counter id. APIs are provided in `counters.h` for convenience; their usage is optional.

## *SST Setup Overview:*

The SST Core has already been compiled and placed in a shared central location. Each student will maintain their own local copy of `sst-elements`. Note: While students are permitted to install SST-core on a different computing platform, it is strongly advised to use the pre-built installation on the HPC cluster, as we will not be able to provide support for alternative setups.

A helper setup script (`init.sh`) and the `sst-elements` source tar file are provided on Moodle.

## *Project Setup Steps:*

1. Navigate to your working directory:
   cd /share/ece506s26/$USER
2. Copy the provided `init.sh` and tar file into this directory.
3. Run one time initial setup:
   setup=1 source init.sh
4. Build the project:
   build=1 source init.sh

You must rebuild (rerun Step 4) after making any changes to the source code (excluding test-examples).

## *Directory Structure:*

- Memory hierarchy elements:
  src/sst-elements/src/sst/elements/memHierarchy/
- L1 coherence manager and counters:
  src/sst-elements/src/sst/elements/memHierarchy/coherencemgr/
- Test cases:
  src/sst-elements/src/sst/elements/memHierarchy/test-examples/

## *Running Tests:*

**Important:** Run all simulations using LSF, not directly on the login node. To run the tests interactively, launch a session with: ‘`bsub -Is -n 1 bash`’.

- Run all test cases:
  python3 run-miranda-bus-coherence-suite.py
- Run a single test case:
  python3 run-miranda-bus-coherence-suite.py <test_name>
- Run with protocol selection:
  MIRANDA_PROTOCOL=MSI python3 run-miranda-bus-coherence-suite.py
  MIRANDA_PROTOCOL=MESI python3 run-miranda-bus-coherence-suite.py

*Submission Instructions:*

Please submit your code for this project through Moodle as a compressed archive (**`<unity_id>_proj2p1.tar.gz`**).

The archive must contain a directory named **`<unity_id>_proj2p1`** which includes the following files:

- The modified MESI_L1.cc file
- Counters.h ( if you made any changes to it)
- The output files generated from running the 7 provided test cases

The expected outputs for all test cases will be provided on Moodle for reference. Your submission will be graded against the provided test cases as well as several unreleased, additional test cases.

# Part 2: Coherence Optimization

The second part of this project requires you to implement a coherence optimization of your choice within the existing L1 coherence controller logic. You can modify any of the existing files within the memHierarchy directory or introduce new ones as needed for your implementation. You must then also demonstrate the resulting improvement using a simple, targeted test case.

The necessary infrastructure, based on the Prospero CPU model, will be provided for writing a test case. Prospero is used because it allows for easy and cycle level controlled tests through text traces that drive multicore memory traffic. However, you are free to use any CPU model from SST for this purpose.

As an example, you may implement a simplified version of the migratory sharing optimization discussed in class.

- Modify the coherence logic in MESI_L1 to enable the optimization
  *(Hint: change how read requests are handled when the optimization is enabled)*
- Illustrate the improvement with a straightforward directed test case. For instance, have all cores sequentially read from and write to the same cache line, thereby forcing line ownership transfers.

You may also choose to implement other coherence optimizations, such as:

- Read snarfing
- Extending MESI to MOESI
- Any other optimization proposed in the literature

## *Project Setup Steps:*

Follow the steps below to set up your environment for this project:

0. Go to the Proj 2.2 assignment on Moodle and download the .tar.gz to a directory on the HPC cluster. Extract the files and move into your SST elements directories as directed below.

1. Replace the following files with the versions provided in the tar archive:

- /memHierarchy/cacheFactory.cc
- /memHierarchy/coherencemgr/MESI_L1.h

2. In your MESI_L1.cc, update the constructor (MESIL1::MESIL1) by adding the following line:

```
coherence_opt_ = params.find<bool>("coherence_opt", false);
```

This flag allows you to enable or disable your optimization logic within MESI_L1.cc. Since the flag is controlled by the test case, you can switch between the base and optimized modes without needing to rebuild.

For example, you can do the following to add code that should only be run when your optimization is enabled:

```
if (coherence_opt_) {
      // your code here
}
```

3. Add Test Infrastructure

- Place the provided test-prospero/ directory inside memHierarchy/
- Refer to memHierarchy/test-prospero/README for detailed instructions on running the tests.

## *Test Infrastructure Overview:*

1. Prospero CPU Test Cases:

- You can add your custom test cases to the provided trace files located in test-prospero/prospero-traces/ ( prospero_core0.trace through prospero_core3.trace).

2. Execution Script:

- A Python script will be provided to run the simulations.This script supports two distinct modes of execution: Base mode and Optimized mode.
    a. Base Mode (Optimized features OFF): Sets OPT_ON=0.
    b. Optimized Mode (Optimized features ON): Sets OPT_ON=1.

### *Submission instructions:*

Please submit your code for this project through Moodle as a compressed archive (**<unity_id>_proj2p2.tar.gz**).

The archive must contain a directory named **<unity_id>_proj2p2** which includes the following files:

- memHierarchy/ directory
  Include your complete modified memHierarchy directory with your directed testcase.
- run.sh script *(if not using Prospero infrastructure)*
  Provide a script to run your test case.
- report.txt
  A simple text file answering the following questions:
    1. Briefly explain the optimization you selected.
    2. What changes did you make to implement the optimization?
    3. Briefly describe your directed test case and the improvement observed.

## Other information and policies

*Please, please* watch the Discord channels for updates. There may be additions or corrections that must be made, and I want everyone to be aware of them. I will not post every single thing to the Announcements forum, but if there is a major change, I will use that to make sure that every student is informed.

This is an ***individual assignment***, not a group project. Do not share your code or your data with others.

*Do not* run simulations on the HPC login nodes. Use a short interactive session (`bsub -Is`) to make sure small simulations are working, but submit **batch jobs** for other needs. There is no need for multicore or exclusive access, so hopefully the delays won't be bad, but (a) we are not the only users of the system, and (b) computing needs will tend to increase as the deadline and the end of the semester approaches. **START EARLY!!!**